Lab 1 - SQL Injection

Lab Setup

We'll be using **SEED Ubuntu-20.04 VM** for this lab. Make sure you've downloaded and installed the VM on you local machine. Follow this page for detailed instructions on how to download and install this VM. Add the following entry in `/etc/hosts` file :

```
10.9.0.5 www.seed-server.com
```

If there's already an entry corresponding to `www.seed-server.com`, remove it and add the above.

Container Setup

The lab uses docker containers. Two containers will be used here, one for hosting a web application and one for the database for the web application. Inside the VM, download the *Labsetup.zip* file from here. Unzip it, enter the Labsetup folder, and use the `docker-compose.yml` file to set up the lab environment. Run `docbuild` and `dockup` consecutively int the same folder as the `docker-compose.yml` to start the containers. Now, you need to enter into the web-application container. For this, run `dockps` and find out the container id of the web-application container. Then execute the command `docksh <container_id>` and you'll get a shell inside the container. Then inside that shell, run the following commands one after another: `cd /etc/apache2/sitesenabled/ rm 000default.conf service apache2 restart`

Now if you hit `10.9.0.5` in firefox, you'll see the vulnerable web application. For more details on working with docker, check this.. As its your very first lab, its highly recommended that you get yourself familiar with the docker commands. You'll need those in upcoming labs.

About the Web Application

We have created a web application, which is a simple employee management application. Employees can view and update their personal information in the database through this web application. There are mainly two roles in this web application: Administrator is a privilege role and can manage each individual employees' profile information; Employee is a normal role and can view or update his/her own profile information.

Tasks

Task 4

We will use the login page from `10.9.0.5` for this task. The login page asks users to provide a user name and a password. The web application authenticate users based on these two pieces of data, so only employees who know their passwords are allowed to log in. Your job, as an attacker, is to log into the web application without knowing any employee's credential.

To help you started with this task, we explain how authentication is implemented in the web application. The PHP code `unsafe_home.php`, located in the `/var/www/SQL_Injection` directory, is used to conduct user authentication. The following code snippet shows how users are authenticated.

```
$input_uname = $_GET['username'];
$input_pwd = $_GET['Password']; $hashed_pwd
= sha1($input_pwd);
...
$sql = "SELECT id, name, eid, salary, birth, ssn, address,
email, nickname, Password
    FROM credential
    WHERE name= '$input_uname' and Password='$hashed_pwd'";
$result = $conn -> query($sql);

// The following is Pseudo Code if(id
!= NULL) { if(name=='admin')
{ return All employees information; }
    else if (name !=NULL){ return
    employee
    information;
```

```
    }
}
else { Authentication Fails;
}
```

The above SQL statement selects personal employee information such as id, name, salary, ssn etc from the `credential` table. The SQL statement uses two variables `input_uname` and `hashed_pwd`, where `input_uname` holds the string typed by users in the username field of the login page, while `hashed_pwd` holds the sha1 hash of the password typed by the user. The program checks whether any record matches with the provided username and password; if there is a match, the user is successfully authenticated, and is given the corresponding employee information. If there is no match, the authentication fails.

Your task is to log into the web application as the very first user in the table which should be **Alice**. But the constraint is you have to assume that *you do not know any of the usernames*. So, you can't use an username directly here. Write the input that you gave in Username and Password field in your report. Also attach the ss after logging in.

Task 5

Assume that you know the number of fields in the `credential` table and also the name of the fields/columns. But you don't know anything about the ordering of the columns. By exploiting the SQL injection vulnerability, how can you determine what's the column number of the `name` field? Write the input that you need to provide in the login page to achieve the mentioned task and also explain why that would work.

Task 6

How can you determine which version of mysql is being used by the web application? And what is the version?

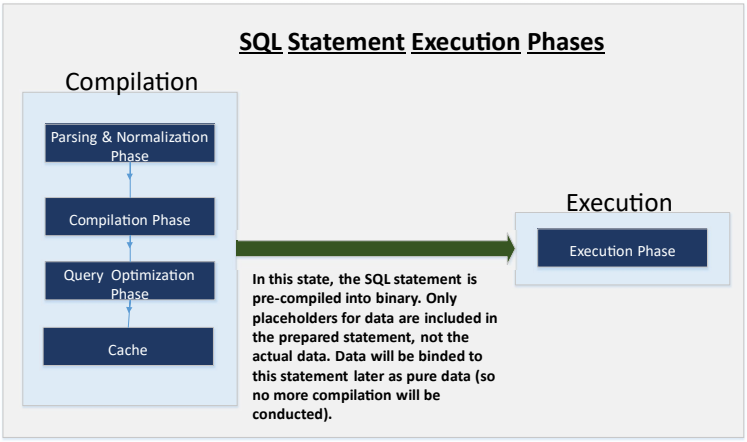

Figure 3: Prepared Statement Workflow

Task 7: Countermeasure — Prepared Statement

The fundamental problem of the SQL injection vulnerability is the failure to separate code from data. When constructing a SQL statement, the program (e.g. PHP program) knows which part is data and which part is code. Unfortunately, when the SQL statement is sent to the database, the boundary has disappeared; the boundaries that the SQL interpreter sees may be different from the original boundaries that was set by the developers. To solve this problem, it is important to ensure that the view of the boundaries are consistent in the server-side code and in the database. The most secure way is to use *prepared statement*.

To understand how prepared statement prevents SQL injection, we need to understand what happens when SQL server receives a query. The high-level workflow of how queries are executed is shown in Figure 3. In the compilation step, queries first go through the parsing and normalization phase, where a query is checked against the syntax and semantics. The next phase is the compilation phase where keywords (e.g. SELECT, FROM, UPDATE, etc.) are converted into a format understandable to machines. Basically, in this phase, query is interpreted. In the query optimization phase, the number of different plans are considered to execute the query, out of which the best optimized plan is chosen. The chosen plan is store in the cache, so whenever the next query comes in, it will be checked against the content in the cache; if it's already present in the cache, the parsing, compilation and query optimization phases will be skipped. The compiled query is then passed to the execution phase where it is actually executed.

Prepared statement comes into the picture after the compilation but before the execution step. A prepared statement will go through the compilation step, and be turned into a pre-compiled query with empty placeholders for data. To run this pre-compiled query, data need to be provided, but these data will not go through the compilation step; instead, they are plugged directly into the precompiled query, and are sent to the execution engine. Therefore, even if there is SQL code inside the data, without going through the compilation step, the code will be simply treated as part of data, without any special meaning. This is how prepared statement prevents SQL injection attacks.

Here is an example of how to write a prepared statement in PHP. We use a SELECT statement in the following example. We show how to use prepared statement to rewrite the code that is vulnerable to SQL injection attacks.

```
$sql = "SELECT name, local, gender
            FROM USER_TABLE
            WHERE id = $id AND
password ='$pwd' "; $result =
$conn->query($sql)
```

The above code is vulnerable to SQL injection attacks. It can be rewritten to the following

```
$stmt = $conn->prepare("SELECT name, local, gender
                                    FROM USER_TABLE
                                    WHERE id =
? and password = ? "); // Bind parameters to
the query
$stmt->bind_param("is", $id, $pwd);
$stmt->execute();
$stmt->bind_result($bind_name, $bind_local,
$bind_gender); $stmt->fetch();
```

Using the prepared statement mechanism, we divide the process of sending a SQL statement to the database into two steps. The first step is to only send the code part, i.e., a SQL statement without the actual the data. This is the prepare step. As we can see from the above code snippet, the actual data are replaced by question marks (?). After this step, we then send the data to the database using bindparam(). The database will treat everything sent in this step only as data, not as code anymore. It binds the data to the corresponding question marks of the prepared statement. In the bindparam() method, the first argument "is" indicates

the types of the parameters: "i" means that the data in $id has the integer type, and "s" means that the data in $pwd has the string type.

Task. In this task, we will use the prepared statement mechanism to fix the SQL injection vulnerabilities. For the sake of simplicity, we created a simplified program inside the defense folder. We will make changes to the files in this folder. If you point your browser to the following URL, you will see a page similar to the login page of the web application. This page allows you to query an employee's information, but you need to provide the correct user name and password.

```
URL: http://www.seed-server.com/defense/
```

The data typed in this page will be sent to the server program getinfo.php, which invokes a program called unsafe.php. The SQL query inside this PHP program is vulnerable to SQL injection attacks. Your job is modify the SQL query in unsafe.php using the prepared statement, so the program can defeat SQL injection attacks. Inside the lab setup folder, the unsafe.php program is in the image_www/Code/ defense folder. You can directly modify the program there. After you are done, you need to rebuild and restart the container, or the changes will not take effect.

You can also modify the file while the container is running. On the running container, the unsafe.php program is inside /var/www/SQL_Injection/defense. The downside of this approach is that in order to keep the docker image small, we have only installed a very simple text editor called nano inside the container. It should be sufficient for simple editing. If you do not like this editor, you can always use "apt install" to install your favoriate command-line editor inside the container. For example, for people who like vim, you can do the following:

```
# apt install -y vim
```

This installation will be discarded after the container is shutdown and destroyed. If you want to make it permanent, add the installation command to the Dockerfile inside the image_www folder.

Submission

You need to submit a detailed lab report, with screenshots, to describe what you have done and what you have observed. Please also list the important code snippets followed by explanation. Simply attaching code without any explanation will not result in full marks.